

MODULE 02

SYLLABUS: Using Transistors and Driving DC Motors: Driving DC Motors with Transistor, Protection Diodes, Controlling Motor Speed with PWM, H-Bridge to Control DC Motor Direction, Roving Robot, Powering Geared Motor with 7805 Regulator, Robot Hardware and Software. Driving Stepper and Servo Motors: Driving Servo Motors, Controlling a Servo, Driving Stepper Motors, Bipolar Stepper Motors Working.

TO WATCH VIDEO CLASSES
OF THIS COURSE: [TAP HERE OR](#)
[SCAN QR CODE](#)



USING TRANSISTORS AND DRIVING DC MOTORS

Introduction to DC Motors

A **DC (Direct Current) motor** is an electromechanical device that converts electrical energy into continuous rotational mechanical motion when a DC voltage is applied across its terminals.

DC motors are found in RC cars, power drills, DVD players, and robotics. Their key controllable properties are:

- **Speed** — controlled by varying the applied voltage (using PWM)
- **Direction** — controlled by reversing the polarity of the voltage (using an H-bridge)
- **Torque** — traded for speed using a gearbox

Brushed DC Motor — Working Principle

A brushed DC motor consists of:

- **Stator** — stationary permanent magnets
- **Rotor (Armature)** — a spinning coil of wire
- **Brushes** — carbon or metal contacts that transfer electrical current to the rotating coil

When current passes through the rotor coil, it generates a magnetic field that is attracted to or repelled by the stator magnets, creating rotational force (torque). The brushes swap the current polarity every half-rotation to sustain continuous rotation.

Generator Effect: If the motor shaft is forcibly turned, the changing magnetic field generates a reverse current back into the circuit. This is why a **protection diode** is always needed when interfacing a DC motor with an Arduino.

Feature	Brushed DC Motor	Brushless DC Motor
Cost	Low	Higher
Speed Control	Easy (PWM)	Complex
Lifespan	Limited (brushes wear)	Longer
Example	RC cars, small robots	Drones, hard drives

Handling High-Current Inductive Loads

DC motors cannot be connected directly to Arduino output pins for two critical reasons:

- 1. **Current requirement:** Arduino I/O pins can supply a maximum of **40 mA** per pin. DC motors typically require hundreds of milliamps to operate — far exceeding this limit.
- 2. **Inductive voltage spikes:** DC motors are inductive loads. When power is suddenly removed, the energy stored in the motor coil is released as a **reverse voltage spike**, which can damage the Arduino or power supply.

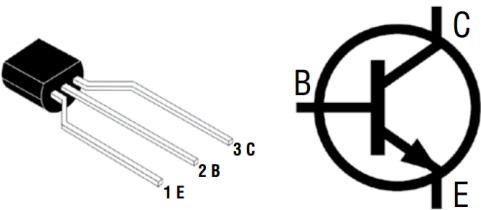
To address these issues, three key components are used: a **transistor** (for switching), a **protection diode** (for spike suppression), and a **separate power supply** (for adequate current).

Using a Transistor as a Switch

NPN Bipolar Junction Transistor (BJT)

A **transistor** is a semiconductor device that acts as an electrically controlled switch. The NPN BJT has three terminals:

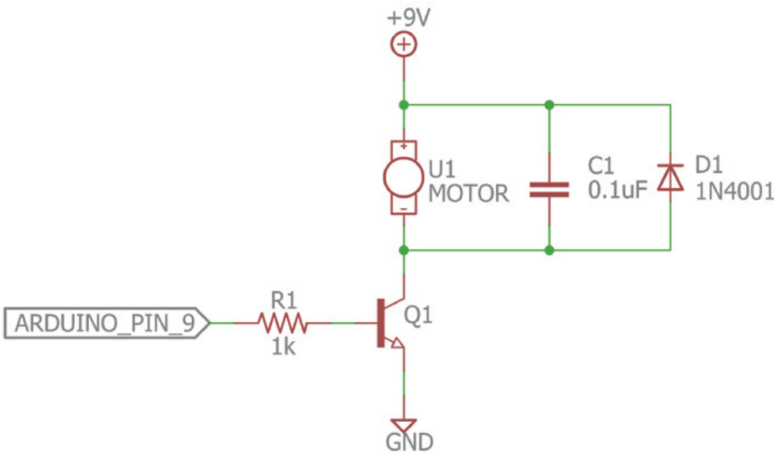
- **Base (B):** Control terminal — a small current/voltage here controls the switch
- **Collector (C):** Current enters through the collector
- **Emitter (E):** Current exits through the emitter



Transistor Operation Modes

Mode	Base Voltage	Behavior	Equivalent
Cut-off	Low / 0V	No current flows C→E	Open switch
Saturation	Sufficient (5V from Arduino)	Current flows freely C→E	Closed switch

When the Arduino sets the control pin HIGH (5V), the base receives sufficient voltage, putting the transistor in **saturation mode** — current flows from the 9V supply through the motor and out of the emitter to ground, spinning the motor. When the pin is LOW, the transistor cuts off and the motor stops.



Component	Role
NPN BJT (PN2222)	Electrically controlled switch — connects/disconnects motor from ground
1kΩ base resistor	Limits current into transistor base from Arduino pin
Protection diode (1N4001)	Absorbs reverse voltage spike from motor inductance
Ceramic capacitor (0.1μF)	Filters electrical noise generated by motor brushes
9V battery	Separate power supply for motor — keeps motor current off Arduino

Protection Diodes (Flyback Diode)

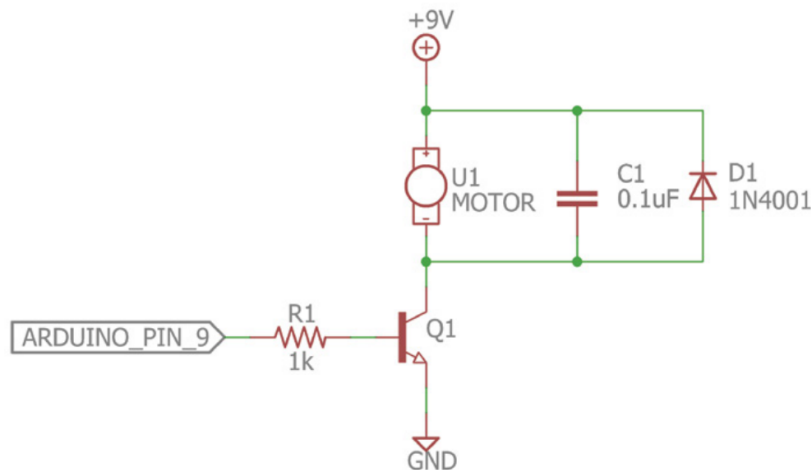
Why It Is Needed

A DC motor is an **inductive load** — it stores energy in the magnetic field of its coils. When the transistor switches the motor off suddenly, this stored energy has nowhere to go. It discharges as a **reverse polarity voltage spike** back toward the power supply and the Arduino. Without protection, this spike can permanently damage the transistor, Arduino, or power supply.

How the Flyback Diode Works

A **flyback diode** (also called a snubber or freewheeling diode) is placed **across the motor terminals** with its cathode toward the positive supply and anode toward the collector side. Since a diode only allows current in one direction:

- During normal operation (motor powered): the diode is reverse-biased — it does not conduct and has no effect on the circuit.
- When the motor is switched off: the reverse voltage spike forward-biases the diode. The spike current safely circulates through the diode loop and dissipates as heat, never reaching the Arduino or power supply.



The stripe on a physical diode marks the **cathode**. Current flows from the side without a stripe (anode) to the side with the stripe (cathode) in normal forward conduction.

Using a Secondary Power Supply

The Arduino's 5V USB supply is insufficient for most motor applications. A **separate 9V battery** is used for the motor circuit.

Key rules:

- Connect the **9V positive** to the motor/transistor collector circuit only.
- The **5V Arduino supply** and **9V motor supply must never be connected together** on the same rail — this can damage the Arduino's voltage regulator.
- The **ground of both supplies must be connected together** to establish a common voltage reference. Without a shared ground, the transistor cannot interpret the Arduino's output pin voltage correctly.

Controlling Motor Speed with PWM

Since the transistor acts as a switch controlled by the Arduino, applying a **PWM signal** to the base rapidly turns the transistor ON and OFF. Due to motor inertia, the motor cannot change speed instantaneously — it averages the PWM switching and runs at a speed proportional to the duty cycle.

Motor Speed Ramp Program

```
const int MOTOR = 9; // Motor control on PWM pin 9

void setup() {
  pinMode(MOTOR, OUTPUT);
}

void loop() {
  // Ramp speed UP: 0 → 255
  for (int i = 0; i < 256; i++) {
    analogWrite(MOTOR, i);
    delay(10);
  }
  delay(2000); // Run at full speed for 2 seconds

  // Ramp speed DOWN: 255 → 0
  for (int i = 255; i >= 0; i--) {
    analogWrite(MOTOR, i);
    delay(10);
  }
  delay(2000); // Stop for 2 seconds
}
```

Motor Speed Control with Potentiometer

```
const int MOTOR = 9; // Motor on PWM pin 9
const int POT = 0; // Potentiometer on Analog pin 0
int val = 0;

void setup() {
  pinMode(MOTOR, OUTPUT);
}

void loop() {
  val = analogRead(POT); // Read pot: 0–1023
  val = map(val, 0, 1023, 0, 255); // Scale to PWM range
  analogWrite(MOTOR, val); // Set motor speed
  // constrain() not needed: pot always returns 0–1023
}
```

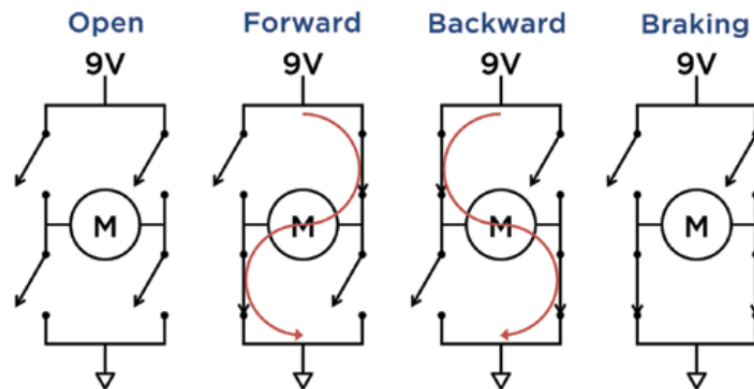
H-Bridge — Controlling DC Motor Direction

Why an H-Bridge is Needed

The transistor switch circuit can only turn the motor ON or OFF — it cannot reverse the motor's direction. Reversing direction requires **reversing the polarity** of the voltage applied to the motor. An **H-bridge** circuit achieves this electronically.

H-Bridge Operation

An H-bridge uses **four switches** (transistors) arranged around a motor. The motor and the four switches form the shape of the letter **H**.



The four operating states:

State	Switches Closed	Current Direction	Motor Behavior
Open	None	No current	Motor free/coasting
Forward	S1 + S4 (diagonal)	Left→Right through motor	Rotates forward
Reverse	S2 + S3 (diagonal)	Right→Left through motor	Rotates backward
Brake	S3 + S4 (both bottom)	Both terminals grounded	Motor stops instantly

Critical Warning: Never close both switches on the same side simultaneously (S1+S3 or S2+S4) — this creates a **direct short circuit** between 9V and GND, which can destroy the H-bridge IC and battery. Always **disable the enable pin before switching direction**.

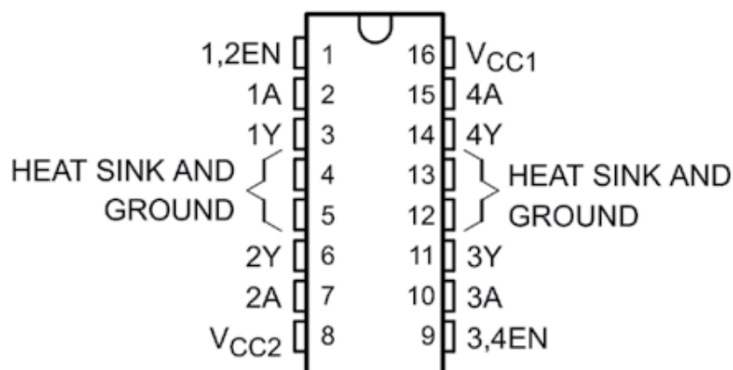
The L293D H-Bridge IC

The **L293D** (Texas Instruments Quadruple Half-H Driver) is the standard H-bridge IC used with Arduino. It contains **two full H-bridges** (four half-H drivers), allowing control of **two DC motors simultaneously**.

Key features:

- Built-in **flyback diodes** — no external protection diodes needed
- Thermal shutdown** protection
- Logic supply (VCC1): 5V; Motor supply (VCC2): up to 36V

L293D Pin Description



FUNCTION TABLE
(each driver)

INPUTS†		OUTPUT
A	EN	Y
H	H	H
L	H	L
X	L	Z

H = high-level, L = low-level
X = irrelevant

Z = high-impedance (off)

† In the thermal shutdown mode, the output is in a high-impedance state regardless of the input levels.

Pin(s)	Name	Connection	Purpose
1	1,2EN	Arduino PWM pin (e.g. pin 9)	Enable/disable left driver; controls speed via PWM
2	1A	Arduino digital pin	Switch 1 control (left driver)
3	1Y	Motor terminal 1	Left driver output 1
4,5,12,13	GND	Common GND	Ground and heat dissipation
6	2Y	Motor terminal 2	Left driver output 2
7	2A	Arduino digital pin	Switch 2 control (left driver)
8	VCC2	9V (motor supply)	Motor power supply
9	3,4EN	GND (if unused)	Enable right driver
16	VCC1	5V (logic supply)	Powers internal logic of IC

H-Bridge Direction Logic Table

1,2EN	1A	2A	Motor Behavior
HIGH	HIGH	LOW	Forward
HIGH	LOW	HIGH	Reverse
LOW	X	X	Motor disabled (coasting)
HIGH	LOW	LOW	Brake

H-Bridge Motor Control Program

```

const int EN = 9; // Enable pin (PWM) → pin 9
const int MC1 = 3; // Motor control 1 → pin 3
const int MC2 = 2; // Motor control 2 → pin 2
const int POT = 0; // Potentiometer → Analog pin 0

int val = 0;
int velocity = 0;

void setup() {
  pinMode(EN, OUTPUT);
  pinMode(MC1, OUTPUT);
  pinMode(MC2, OUTPUT);
  brake(); // Always initialize with motor stopped
}

// Move motor FORWARD at given rate (0–255)
void forward(int rate) {
  digitalWrite(EN, LOW); // Disable first — prevent short
  digitalWrite(MC1, HIGH); // Set direction: forward
  digitalWrite(MC2, LOW);
  analogWrite(EN, rate); // Re-enable with PWM speed
}

```

```

// Move motor BACKWARD at given rate (0–255)
void reverse(int rate) {
  digitalWrite(EN, LOW); // Disable first
  digitalWrite(MC1, LOW); // Set direction: reverse
  digitalWrite(MC2, HIGH);
  analogWrite(EN, rate); // Re-enable with PWM speed
}

// Stop motor immediately
void brake() {
  digitalWrite(EN, LOW); // Disable
  digitalWrite(MC1, LOW); // Both control pins LOW = brake
  digitalWrite(MC2, LOW);
  digitalWrite(EN, HIGH); // Re-enable to hold brake state
}

void loop() {
  val = analogRead(POT); // Read potentiometer

  if (val > 562) { // Upper half → Forward
    velocity = map(val, 563, 1023, 0, 255);
    forward(velocity);
  }
  else if (val < 462) { // Lower half → Reverse
    velocity = map(val, 461, 0, 0, 255); // Inverted mapping
    reverse(velocity);
  }
  else { // Dead zone (462–562) → Brake
    brake();
  }
}

```



Key programming concepts:

- Functions are always called with EN set LOW first — this prevents accidental short circuits when switching direction.
- `map(val, 461, 0, 0, 255)` — passing arguments in descending order inverts the mapping, so that moving the pot toward 0 increases reverse speed.

The Roving Robot

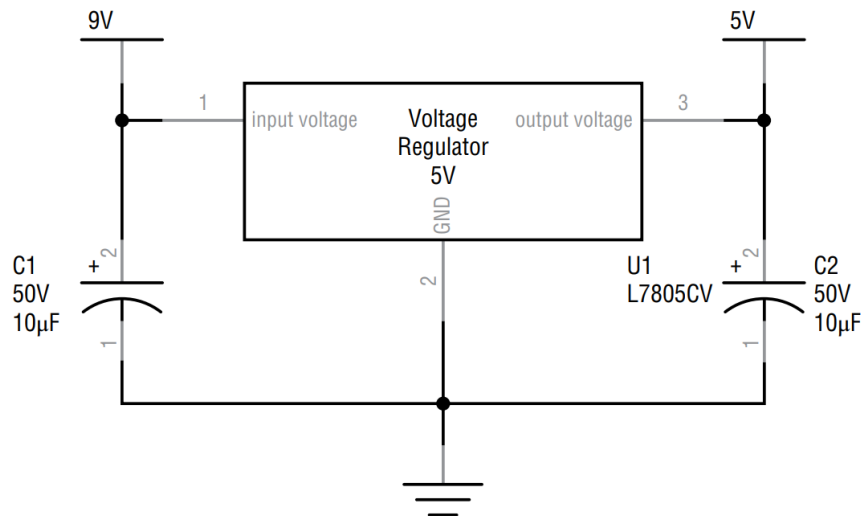
Concept

A light-following robot uses two **photoresistors** as sensors and two **geared DC motors** driven by an L293D H-bridge. The robot steers toward the brightest light source:

- Light on the **right sensor** → **left motor** speeds up → robot turns right (toward light)
- Light on the **left sensor** → **right motor** speeds up → robot turns left (toward light)
- Equal light on both → both motors run at equal speed → robot goes straight

Powering Geared Motors with the L7805CV Voltage Regulator

Geared DC motors for robots operate at 5V but draw up to 550–650 mA stall current — more than the Arduino's onboard 5V regulator can supply. The solution is a **separate 5V supply** generated using the **L7805CV linear voltage regulator**.



L7805CV Wiring (with flat metal tab facing away):

Pin Position	Signal	Connection
Leftmost	Input	9V battery
Center	GND	Common ground
Rightmost	Output	5V motor rail

The 5V motor rail from the regulator must be kept **separate** from the Arduino's 5V pin. Only the **GND must be shared** between all supplies and the Arduino.

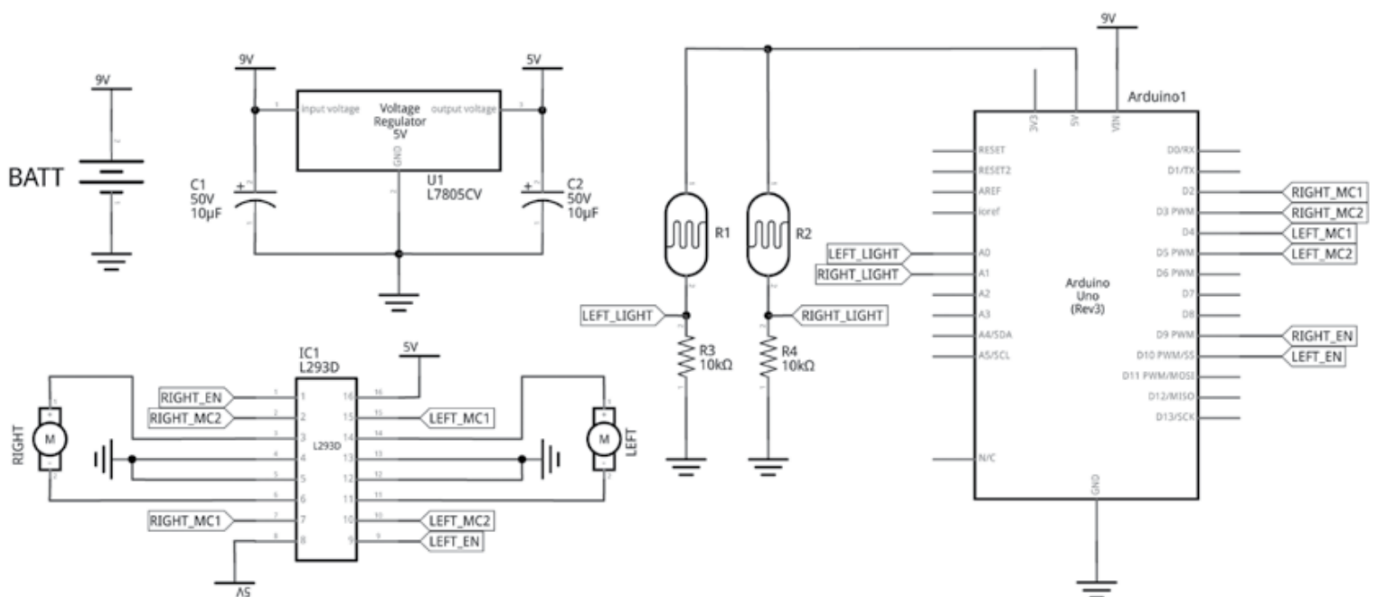


Fig: Roving robot schematic

Robot Software — Key Logic

```
// Key constants (calibrate using Serial Monitor)
const int LIGHT_THRESHOLD_MIN = 810; // Min ADC value to trigger movement
const int LIGHT_THRESHOLD_MAX = 1100;
const int SPEED_MIN = 150;
const int SPEED_MAX = 255;

void loop() {
  int left_light = analogRead(LEFT_LIGHT_SENSOR);
  int right_light = analogRead(RIGHT_LIGHT_SENSOR);

  // Right sensor → controls LEFT motor
  if (right_light >= LIGHT_THRESHOLD_MIN) {
    int left_speed = map(right_light,
                        LIGHT_THRESHOLD_MIN, LIGHT_THRESHOLD_MAX,
                        SPEED_MIN, SPEED_MAX);
    left_speed = constrain(left_speed, SPEED_MIN, SPEED_MAX);
    forward("left", left_speed);
  } else {
    brake("left");
  }

  // Left sensor → controls RIGHT motor
  if (left_light >= LIGHT_THRESHOLD_MIN) {
    int right_speed = map(left_light,
                        LIGHT_THRESHOLD_MIN, LIGHT_THRESHOLD_MAX,
                        SPEED_MIN, SPEED_MAX);
    right_speed = constrain(right_speed, SPEED_MIN, SPEED_MAX);
    forward("right", right_speed);
  } else {
    brake("right");
  }
}
```

Calibration procedure: Upload the sketch, open Serial Monitor, shine a flashlight at each photoresistor and record the minimum and maximum ADC values observed. Set these as `LIGHT_THRESHOLD_MIN` and `LIGHT_THRESHOLD_MAX` in the code.

DRIVING STEPPER AND SERVO MOTORS

Introduction to Servo Motors

A **servo motor** is a special type of motor designed for **precise angular position control**. Unlike DC motors that spin continuously, a servo rotates to a specific commanded angle and holds that position until instructed otherwise. This makes servos ideal for applications requiring known, repeatable positioning — door locks, robotic arms, RC airplane control surfaces, and aperture control systems.

Standard vs Continuous Rotation Servos

Feature	Standard Servo	Continuous Rotation Servo
Range	Fixed: 0° to 180°	360° continuous
Control	Pulse length sets absolute position	Pulse length sets speed and direction
Internal pot	Present (position feedback)	Removed or bypassed
Use case	Robotic arms, steering, locks	Wheels, conveyor drives

A standard servo contains an internal **potentiometer** mechanically linked to the output shaft. This potentiometer provides position feedback to the servo's internal control circuit, allowing it to rotate to and hold a precise angle. Removing this potentiometer converts it to a continuous rotation servo.

Servo Motor Pins and Wiring

Unlike a DC motor with only two power terminals, a servo has **three wires**:

Wire Color	Signal	Connection
Red	Power (VCC)	5V supply (from external regulator)
Black / Brown	Ground	Common GND
White / Orange	Control signal	Arduino digital pin (e.g. pin 9)

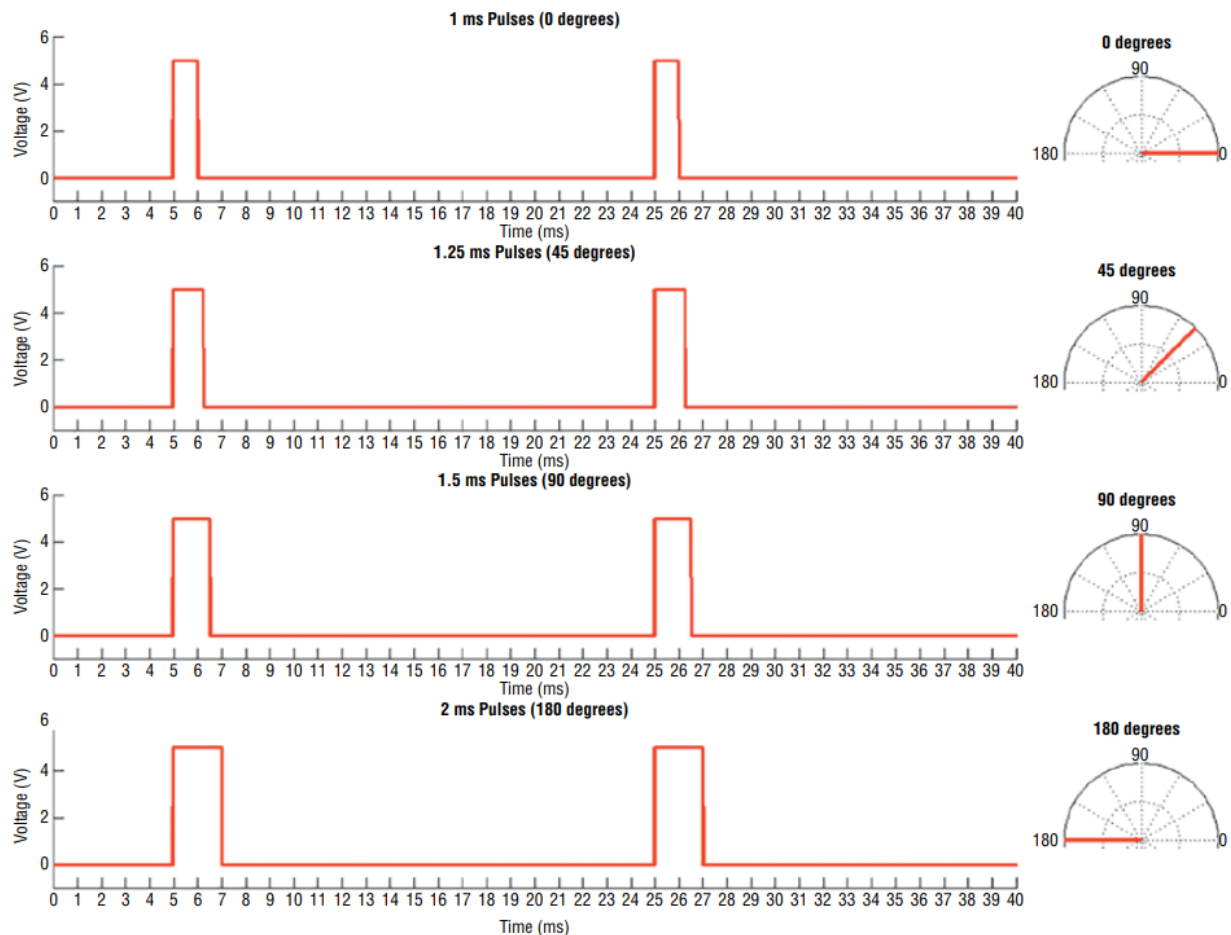
Always verify wire colors against the specific servo's datasheet — color coding may vary between manufacturers.

Power consideration: Servos can draw significant current — often more than one Arduino pin or the Arduino's onboard 5V regulator can supply, especially under load. Use the same **L7805CV 5V regulator circuit** from the DC motor chapter to provide a separate, stable 5V supply for the servo. The control signal wire connects to the Arduino pin directly. The grounds of both supplies must be tied together.

Servo Control — Pulse Width Signaling

Servos are not controlled by simple HIGH/LOW signals or PWM duty cycles the way LEDs and motors are. They use a specific **pulse width modulation timing protocol**:

- A pulse is sent on the control wire **once every 20 ms**.
- The **length (width) of the pulse** determines the angle:
 - **1 ms pulse** → 0°
 - **1.5 ms pulse** → 90°
 - **2 ms pulse** → 180°
 - Values between 1 ms and 2 ms scale linearly to intermediate angles.
- To **hold** a position (resist external force), the same pulse is resent every 20 ms continuously.



Pulse Width to Angle Mapping

Pulse Width	Servo Angle
1.0 ms	0°
1.25 ms	45°
1.5 ms	90°
1.75 ms	135°
2.0 ms	180°

Controlling a Servo with Arduino — The Servo Library

Rather than manually generating the precise 1–2 ms pulses every 20 ms, the Arduino IDE provides a built-in **Servo library** that handles all timing automatically. The programmer only needs to specify the target angle.

Including and Using the Servo Library

```
#include <Servo.h>    // Import the Servo library
Servo myServo;        // Create a Servo object named myServo
```

- `#include <Servo.h>` adds the servo library's functions to the sketch.
- `Servo myServo` creates a servo **object** — a named instance that represents the physical servo. Multiple servos can be created with different names.

Key Servo Library Functions

Function	Purpose
myServo.attach(pin)	Initializes the servo on the specified pin; sets it as output automatically
myServo.write(angle)	Sends a pulse to rotate servo to given angle (0–179°)

Servo Potentiometer Control Program

```
#include <Servo.h>

const int SERVO = 9; // Servo control wire → pin 9
const int POT = 0; // Potentiometer → Analog pin 0

Servo myServo; // Create servo object
int val = 0;

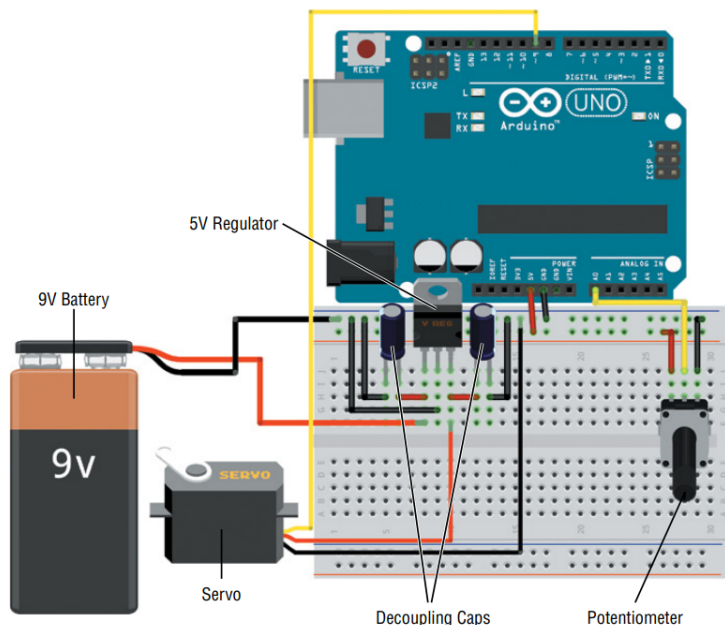
void setup() {
  myServo.attach(SERVO); // Attach servo to pin 9
  // No pinMode needed — attach() handles it
}

void loop() {
  val = analogRead(POT); // Read pot: 0–1023
  val = map(val, 0, 1023, 0, 179); // Scale to servo range: 0–179°
  myServo.write(val); // Command servo to that angle
  delay(15); // Allow servo time to reach position
}
```

Program flow: The potentiometer's ADC value (0–1023) is mapped to the servo's angular range (0–179°). At pot position 0, servo moves to 0°; at full rotation, servo moves to 179°. The 15 ms delay allows the servo to physically reach the commanded angle before the next command is sent.

Important: When the Servo library is active, **PWM output on pins 9 and 10 is unavailable** on the Uno. This is because the servo library uses the same hardware timer (Timer1) that generates PWM on those pins. Use PWM pins 3, 5, 6, or 11 for other PWM outputs when a servo is in use.

Wiring Diagram — Servo with 5V Regulator



Introduction to Stepper Motors

A **stepper motor** is a brushless DC motor that moves in discrete, precise angular increments called **steps**. Instead of spinning continuously like a DC motor, a stepper advances one step at a time each time a coil is energized. The step size is fixed and determined by the motor's internal mechanical design.

Why Stepper Motors Are Used

Feature	Stepper Motor	DC Motor
Position control	Precise (open-loop)	None without encoder
Speed at low RPM	High torque	Low torque
Feedback needed	No	Yes, for position
Applications	3D printers, CNC, instruments	Fans, wheels, drills

Stepper motors are preferred wherever **accurate, repeatable positioning** is required without a feedback sensor. Common applications include 3D printer gantries, CNC machines, instrument panel gauges, and robotic joints.

Types of Stepper Motors

Type	Description	Torque	Driver Complexity
Unipolar	Each phase energized in one current direction only	Lower (half torque)	Simpler
Bipolar	Each phase energized in both current directions	Full torque	Needs H-bridge

This course uses **bipolar stepper motors** (specifically the NEMA-17), which deliver maximum torque by driving each coil in both directions using an H-bridge.

NEMA-17 refers to the standardized mounting face size (1.7 inches × 1.7 inches) — it does not specify the motor's electrical characteristics. NEMA-17 motors are available in a wide range of torque and step angle ratings.

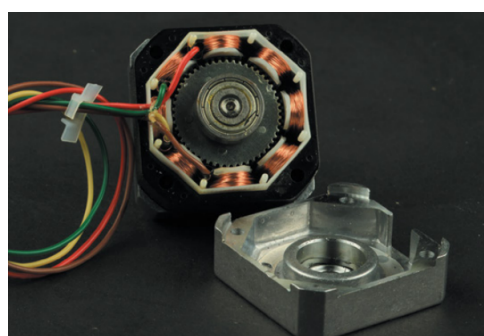
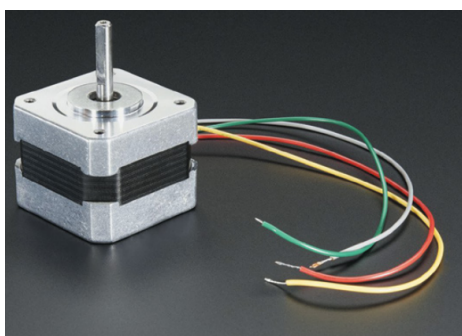
Bipolar Stepper Motor — Internal Construction

A bipolar stepper motor contains:

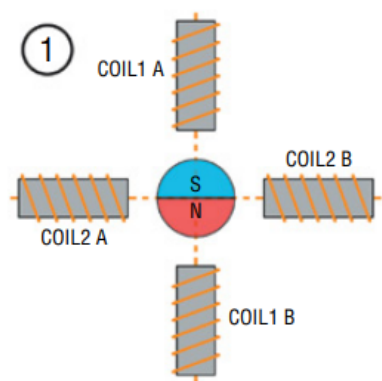
- **Two phases (COIL 1 and COIL 2)** — each made of multiple wire coils wound around soft metal cores
- **Four wire leads** — two per phase (COIL1_A, COIL1_B, COIL2_A, COIL2_B)
- **Central rotating permanent magnet (rotor)** with alternating N-S poles

Each coil, when energized with current, becomes an **electromagnet**. The rotating permanent magnet aligns itself with the nearest active electromagnet. By energizing the coils in a specific sequence, the rotor is pulled around in a controlled, stepping motion.

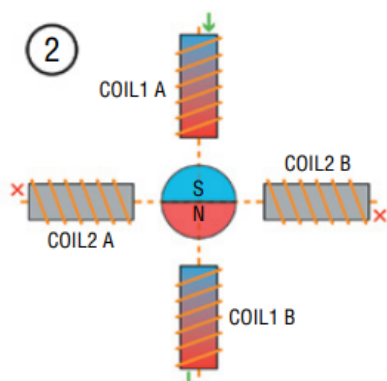
The NEMA-17 motor achieves **200 steps per revolution** (1.8° per step) by having multiple coil pairs and a multi-pole rotor magnet.



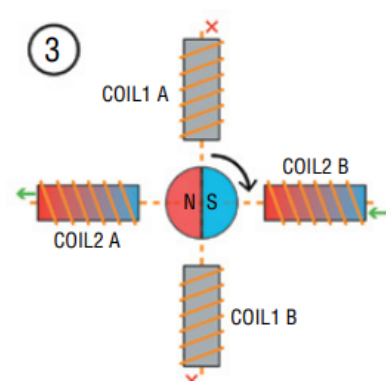
Bipolar Stepper Motor — Step Sequence (Working)



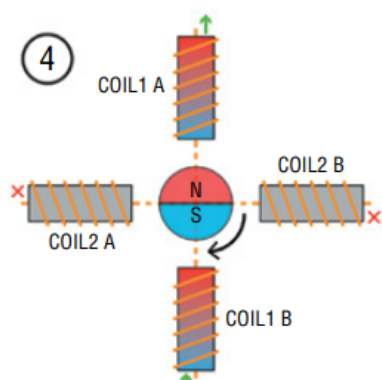
Not Energized
No current flowing through coils.
Rotor can spin freely.



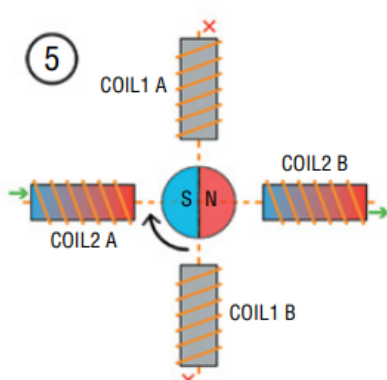
Energized
Current flows through coil 1, inducing a magnetic field. Rotor locks in position.



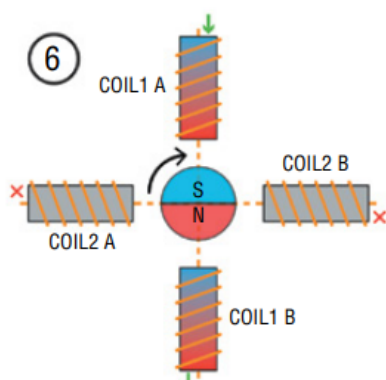
Energized, Rotating
Current flows through coil 2, inducing a new magnetic field. Permanent magnet (shaft) rotates to align to it.



Energized, Rotating
Current flows through coil 1 again, in opposite direction from step 2. Magnetic field direction now opposite of step 2. Shaft rotates to align to it.



Energized, Rotating
Current flows through coil 2, again, in opposite direction from step 3. Magnetic field direction now opposite of step 3. Shaft rotates to align to it.



Energized, Rotating
Step 2 repeats. Shaft rotates again, returning to starting position.

The stepper advances through the following 4-step sequence (which repeats):

Step	COIL 1	COIL 2	Effect
1 (Neutral)	Off	Off	No magnetic field — rotor spins freely
2	Current: A→B	Off	Magnetic field locks rotor in position
3	Off	Current: B→A	Field rotates 90° — rotor follows (step 1)
4	Current: B→A (reversed)	Off	Field reverses — rotor moves another step
5	Off	Current: A→B (reversed)	Field rotates again — rotor follows
6	Repeat Step 2	—	Rotor has completed full cycle

Each time a coil is energized in a new direction, the rotor's permanent magnet is attracted to the new magnetic field and rotates to align with it. Reversing the current through a coil reverses the magnetic field direction — this requires an H-bridge for each phase.

Practical note: The simplified Fig (above shown) shows a 4-step motor. The real NEMA-17 has 200 steps per revolution because it has more coil pairs and a multi-pole rotor — the principle is identical, just scaled up.

Driving a Stepper Motor — Hardware

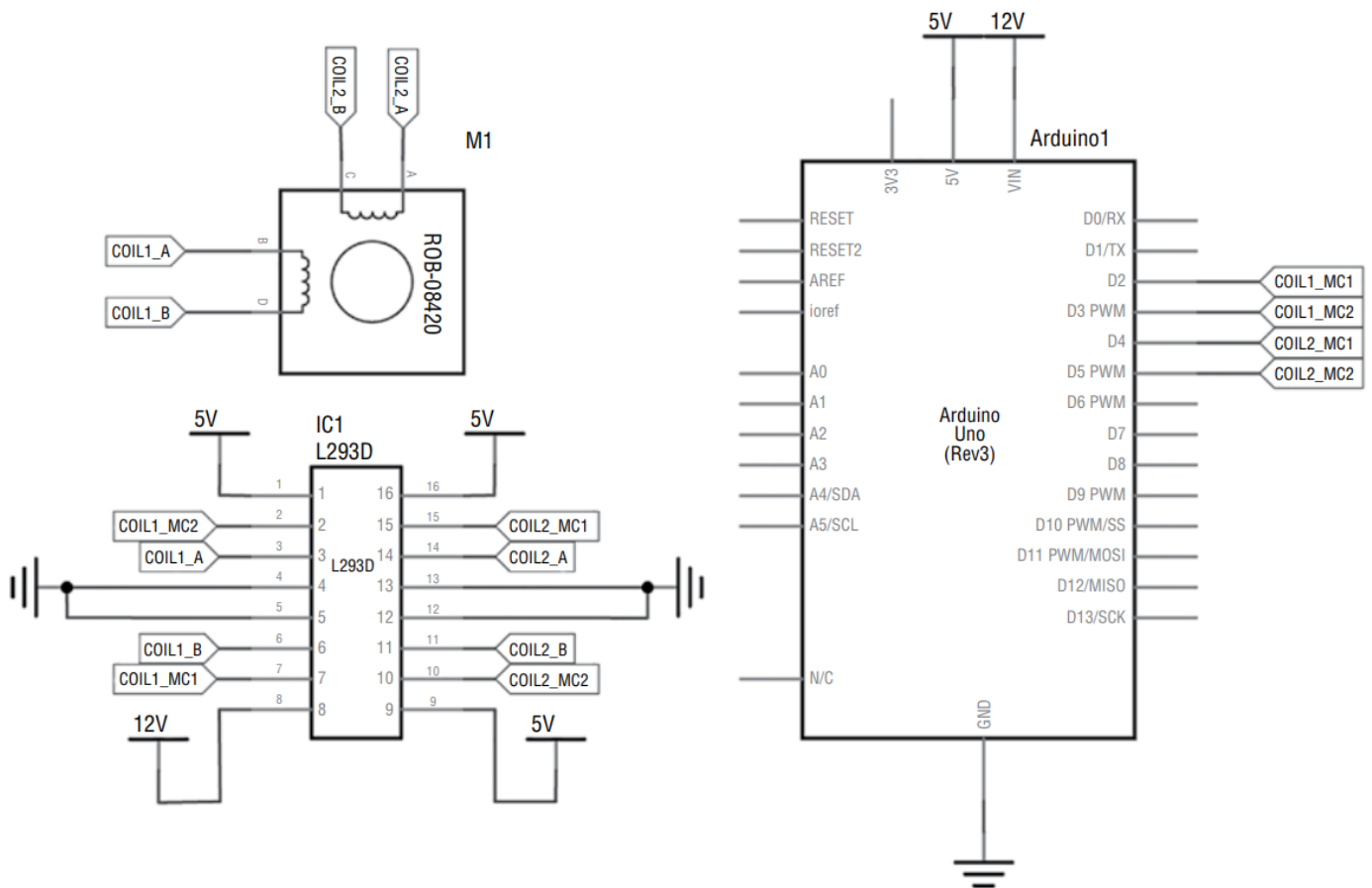
Since a bipolar stepper requires **two bidirectional H-bridges** (one per phase), the same **L293D** IC used for DC motors is reused here — both full H-bridge drivers are now used, one for each coil.

Power Supply

The NEMA-17 stepper requires **12V** to operate at rated torque. A **12V DC wall adapter** (>500 mA) is plugged into the Arduino's barrel jack. This provides:

- 12V on the **VIN pin** → connected to L293D pin 8 (VCC2, motor supply)
- Arduino's onboard regulator generates 5V → connected to L293D pin 16 (VCC1, logic supply)

The enable pins (1,2EN and 3,4EN) are tied **permanently to 5V** (always enabled) — the stepper library handles coil sequencing safely, so no risk of H-bridge shorts.



Identifying Stepper Motor Coil Pairs

If wire-to-coil labeling is unknown, use a **multimeter in resistance/continuity mode**:

- Two wires from the **same phase** will show low resistance ($<10\ \Omega$) between them.
- Two wires from **different phases** will show no continuity (open circuit).
- Once pairs are identified, either wire of a pair can go to either pin of the same H-bridge — reversing them only reverses the default rotation direction.

Stepper Motor Control — The Stepper Library

The Arduino IDE includes a built-in **Stepper library** that handles the precise coil energizing sequence automatically.

Key Stepper Library Functions

Function	Purpose
Stepper myStepper(steps, pin1, pin2, pin3, pin4)	Creates stepper object; defines steps/rev and 4 control pins
myStepper.setSpeed(rpm)	Sets motor speed in rotations per minute
myStepper.step(n)	Moves motor n steps; negative n reverses direction; blocking call

step() is a **blocking function** — the program waits until all requested steps are completed before executing the next line of code.

Simple Stepper Back-and-Forth Program

```
#include <Stepper.h> // Import Stepper library

const int STEPS_PER_REV = 200; // NEMA-17: 200 steps = 1 full rotation

// L293D control pins for coil switching
const int COIL1_MC1 = 2;
const int COIL1_MC2 = 3;
const int COIL2_MC1 = 4;
const int COIL2_MC2 = 5;

// Create stepper object: steps/rev + 4 H-bridge control pins
Stepper myStepper(STEPS_PER_REV, COIL1_MC1, COIL1_MC2,
                  COIL2_MC1, COIL2_MC2);

void setup() {
  myStepper.setSpeed(60); // Set speed to 60 RPM
}

void loop() {
  myStepper.step(STEPS_PER_REV); // One full rotation forward
  delay(500);
  myStepper.step(-STEPS_PER_REV); // One full rotation backward
  delay(500);
}
```

Application — One-Minute Chronograph

A chronograph (precision timer) uses the stepper motor's fixed step size to create an accurately timed sweep. Since the NEMA-17 has exactly 200 steps per revolution, and one full rotation must complete in exactly 60 seconds:

$$\text{Time per step} = 60,000 \text{ ms} \div 200 \text{ steps} = 300 \text{ ms/step}$$

The motor advances exactly one step every 300 ms, completing one full sweep in precisely 60 seconds. This would be impossible with a brushed DC motor, which has no built-in position feedback.

millis() Function

unsigned long millis();

Returns the number of milliseconds elapsed since the Arduino started running. Used here to implement **non-blocking timing** — the program checks elapsed time each loop iteration rather than using delay(), which would block button reading.

Variable type must be unsigned long (holds large positive numbers without overflow).

Internal Pull-Up Resistors for Buttons

Rather than using external 10kΩ pull-up resistors, the ATmega328P's built-in pull-ups are activated:

```
pinMode(START, INPUT_PULLUP);
```

```
pinMode(STOP, INPUT_PULLUP);
```

With INPUT_PULLUP:

- Default (button open) → pin reads **HIGH**
- Button pressed (shorts pin to GND) → pin reads **LOW**

This simplifies wiring — only the button itself (connecting pin to GND when pressed) is needed. No external resistor required.

Limitation: Internal pull-up is only active after the bootloader has run and the code has executed pinMode(). If the connected device cannot tolerate a floating pin during the brief bootup period, use an external pull-up resistor instead.

Chronograph Program

```
#include <Stepper.h>

const int STEPS_PER_REV = 200; // 200 steps = 1 revolution
const int MS_PER_STEP = 300; // 60000ms / 200 steps = 300ms/step

const int COIL1_MC1 = 2;
const int COIL1_MC2 = 3;
const int COIL2_MC1 = 4;
const int COIL2_MC2 = 5;

const int START = 8; // Start button → pin 8
const int STOP = 9; // Stop button → pin 9

// Time tracking variables (unsigned long for large ms values)
unsigned long last_time = 0;
unsigned long curr_time = 0;
int steps_taken = 0;

Stepper chronograph(STEPS_PER_REV, COIL1_MC1, COIL1_MC2,
                    COIL2_MC1, COIL2_MC2);

void setup() {
  chronograph.setSpeed(200); // High speed for fast reset sweep

  // Enable internal pull-up resistors on button pins
  pinMode(START, INPUT_PULLUP);
  pinMode(STOP, INPUT_PULLUP);
}
```

```

void loop() {
  // Step 1: Wait here until START button is pressed (goes LOW)
  while(digitalRead(START) == HIGH); // Semicolon = empty loop body
                                     // Stays here until START pressed

  last_time = millis(); // Record start time

  // Step 2: Run until STOP pressed OR full revolution complete
  while(digitalRead(STOP) == HIGH && steps_taken < STEPS_PER_REV) {
    curr_time = millis();

    // Step 3: Every 300ms, advance one step
    if(curr_time - last_time >= MS_PER_STEP) {
      chronograph.step(1); // Advance one step
      steps_taken++;       // Count steps taken
      last_time = curr_time; // Reset timer reference
    }
  }

  // Step 4: If stopped early, return dial to start position
  if(steps_taken < STEPS_PER_REV) {
    chronograph.step(-steps_taken); // Reverse back to start
  }

  steps_taken = 0; // Reset counter for next run
}

```

Program logic breakdown:

Code Element	Purpose
while(digitalRead(START)==HIGH);	Blocking wait — holds until START pressed; semicolon means empty body
millis()	Returns elapsed ms since Arduino boot — used for non-blocking timing
curr_time - last_time >= MS_PER_STEP	Checks if 300ms have elapsed since last step
chronograph.step(1)	Advances exactly 1 step
steps_taken++	Counts steps for position tracking and reset
chronograph.step(-steps_taken)	Returns to start if stopped early
unsigned long	Required type for millis() — holds large positive ms values

Comparison — DC Motor, Servo Motor, and Stepper Motor

Feature	DC Motor	Servo Motor	Stepper Motor
Motion type	Continuous rotation	Angular position (0–180°)	Discrete steps
Position control	No (needs encoder)	Yes (internal pot)	Yes (open-loop)
Speed control	PWM	Not primary purpose	setSpeed() in RPM
Direction control	H-bridge	Pulse width	Coil sequence reversal
Driver needed	Transistor / H-bridge	None (built-in)	H-bridge (2 channels)
Torque at low speed	Low	High (geared)	High
Typical use	Wheels, fans, drills	Robotic arms, RC, locks	3D printers, CNC, gauges
Arduino library	None (analogWrite)	Servo.h	Stepper.h

MODEL QUESTIONS

1. Draw the PWM signal needed to rotate a servo motor to 45° (3 Marks)
2. Draw the circuit for interfacing a DC motor to Arduino? What is the need for the protection diode? (3 Marks)
3. Write code to control the speed of a DC motor with the Arduino. Comment your code for easy understanding. (4 Marks)
4. Explain the working a 4-steps per rotation bipolar stepper motor with diagrams. (5 Marks)
5. Explain the working of an H-bridge circuit. Explain the features of L293D IC. (4 Marks)
6. Write code to play a tone with the frequency set by a variable resistor connected to analog input. Comment your code for improving readability. (5 Marks)